LATINOWARE 2026

23º Congresso Latino-americano de Software Livre e Tecnologias Abertas

Realização :

itaipu parquetec

ITAIPU BINACIONAL

Latin.Science 2026

Open Source LVC Simulation Architecture

Arquitetura event-driven Open Source, com Event Sourcing e CQRS, para simulações LVC em Smart Cities.

Autores:

Rafael Rodrigues Pereira
Luiza Ribeiro dos Santos
Marcos Vogel
Bruno da Silva Rodrigues
Fabio Silva Lopes
Fabio Kawaoka Takase

Agenda

01 Problema e lacuna

02 Arquitetura proposta

03 Cenários e avaliação

04 Contribuições e limites

01

PRÓXIMA SEÇÃO

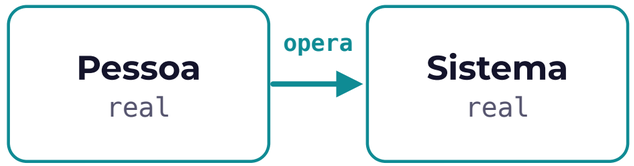
Problema e lacuna

LVC: quem é real e quem é simulado.

real | simulado

LIVE

Pessoa real, sistema real

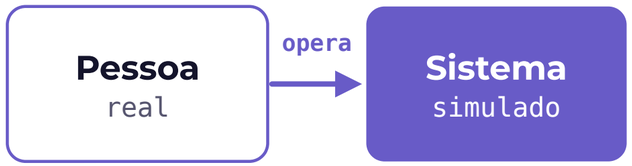


Ônibus, navios e alagamentos de verdade, vindos de fontes públicas.

no projeto · AIS · OlhoVivo · CGESP

VIRTUAL

Pessoa real, sistema simulado

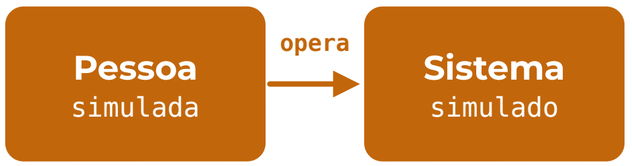


Um operador real injeta eventos no gêmeo digital do campus.

no projeto · virtual-ms-java · Vaadin

CONSTRUCTIVE

Pessoa e sistema simulados



Um drone autônomo no AirSim, com missão planejada por LLM.

no projeto · constructive-ms-python

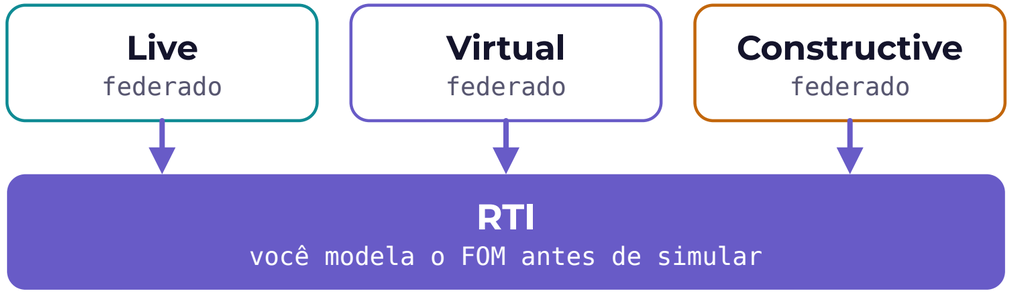
Para os três conversarem, falta um formato de mensagem comum e um canal entre os serviços.

HLA exige RTI e FOM; DIS usa PDUs fixas.

Latin.Science 2026

HLA · IEEE 1516

RTI central + FOM + gestão de tempo

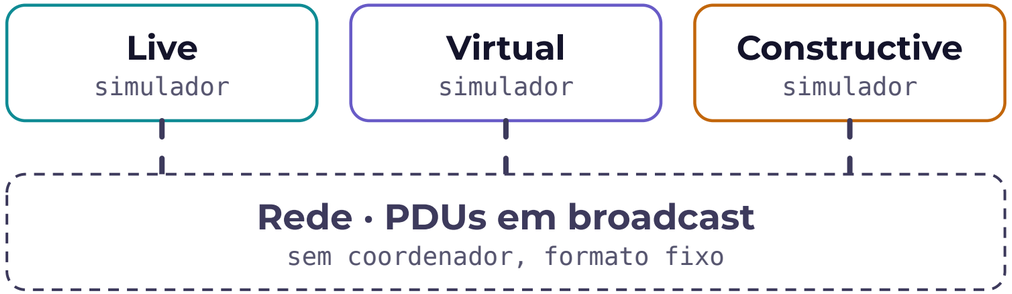


\+ Padrão IEEE consolidado

\- **Exige modelar o FOM e operar uma RTI**

DIS · IEEE 1278

PDUs fixas em broadcast, sem RTI



\+ Simples de ligar

\- **Modelo de dados rígido, nascido na defesa**

PERGUNTA **Dá para integrar LVC numa arquitetura aberta, rastreável e reproduzível?**

Lacuna · Mazzetto (2024): gêmeos urbanos esbarram em barreiras de integração e pedem padrões comuns.

02

PRÓXIMA SEÇÃO

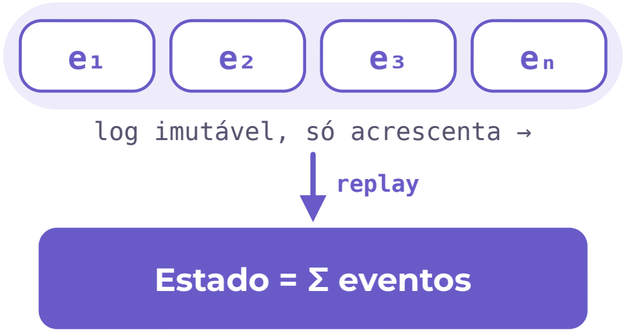
Arquitetura proposta

Event Sourcing, CQRS e um .proto único.

Sistemas LVC são **reativos por natureza**: navios, ônibus, alertas, missões.

EVENT SOURCING

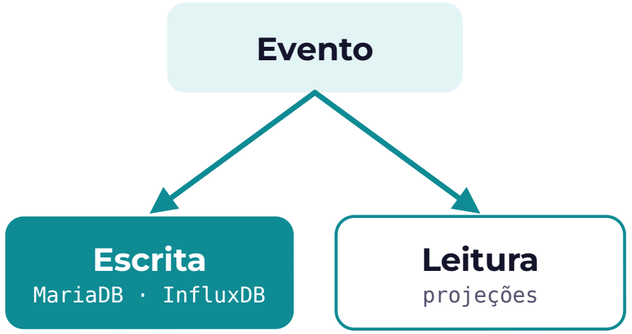
Estado reconstruído por replay



Cada mudança vira um evento imutável; o replay reconstrói qualquer instante.

CQRS

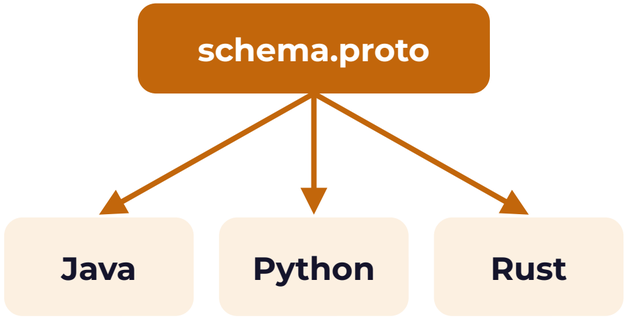
Escrita separada da leitura



Escrita de alta frequência separada das projeções que servem os dashboards.

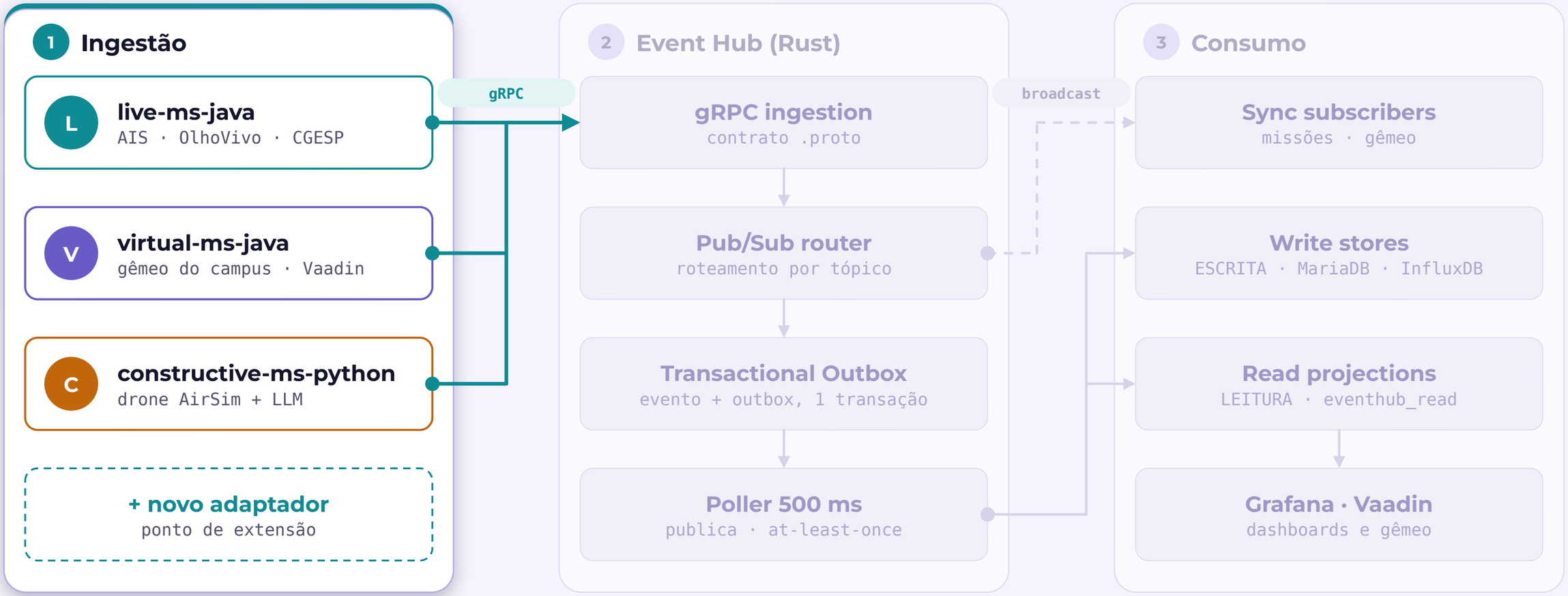
CONTRATO CANÔNICO

Um .proto, 3 linguagens



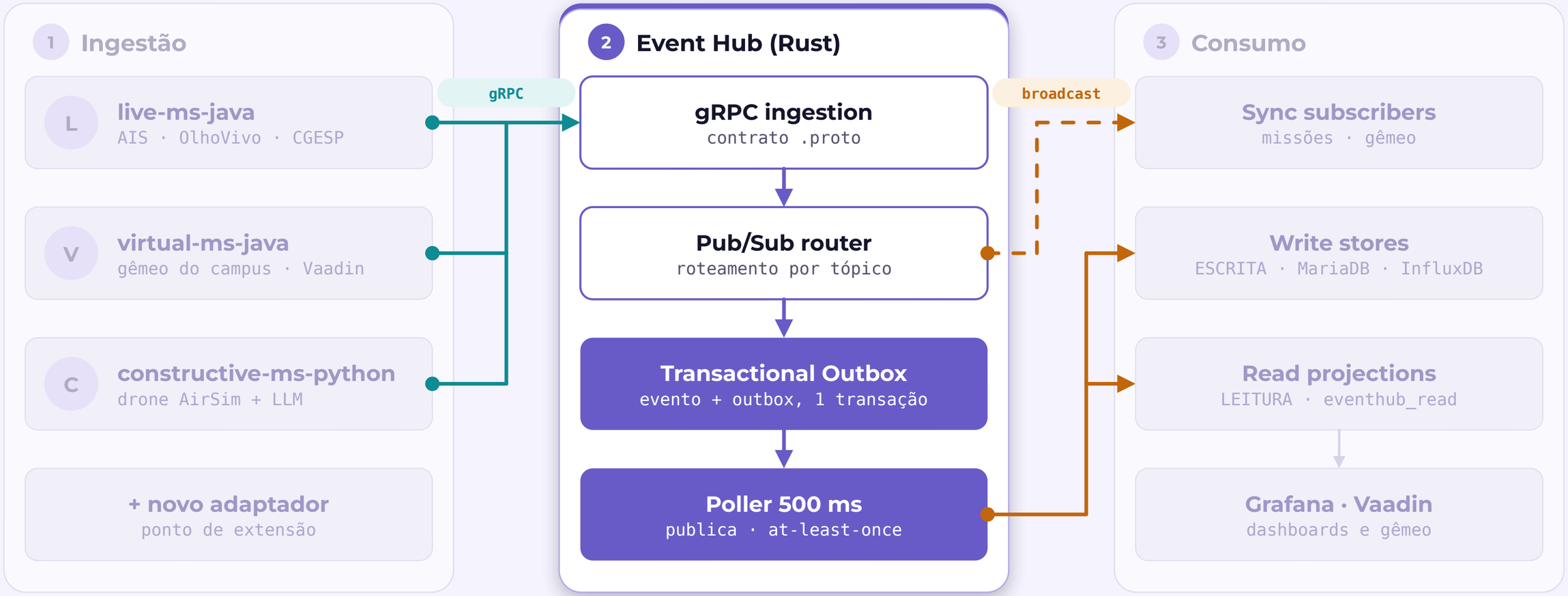
O proto-shared gera os stubs de Java, Python e Rust a partir de um arquivo.

Ingestão: três serviços publicam via gRPC no .proto.



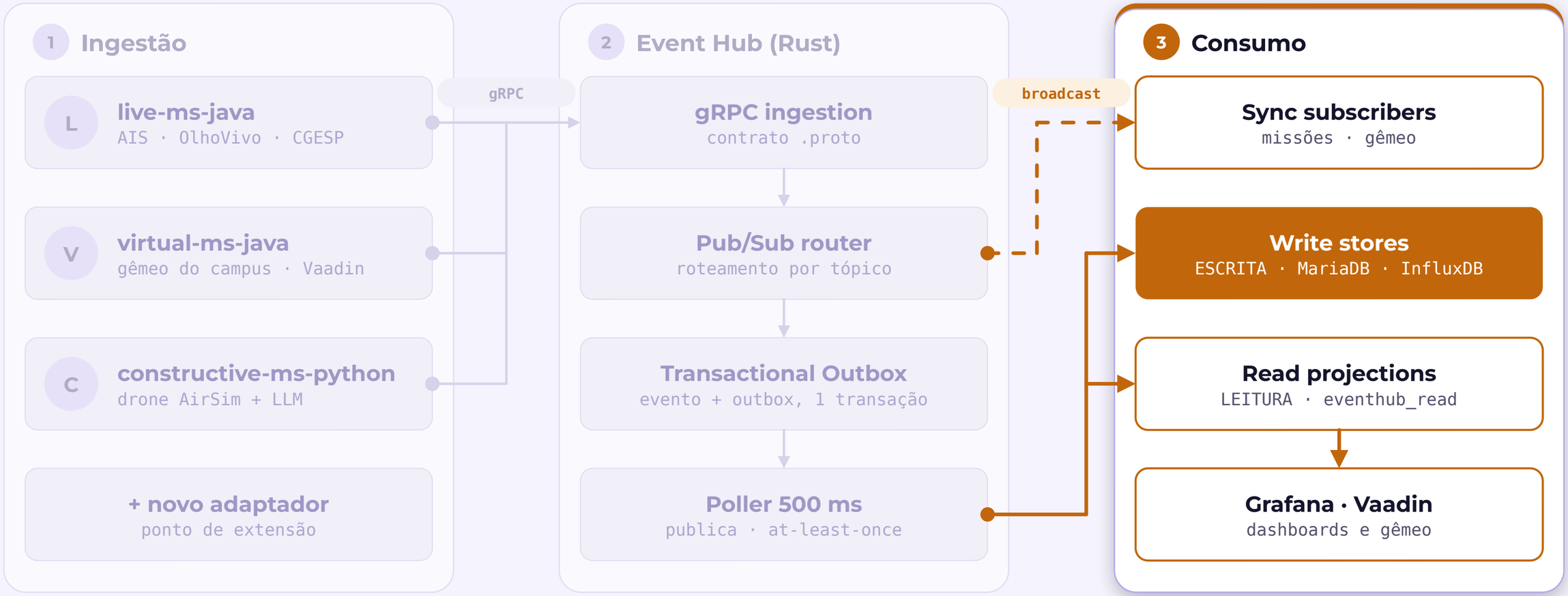
CAMADA 1 Um contrato .proto, via gRPC, para os três serviços; adaptador novo é só mais um produtor.

Event Hub: outbox transacional e poller de 500 ms.



CAMADA 2 Evento e outbox na mesma transação local; o poller publica a cada 500 ms: entrega at-least-once.

Consumo: MariaDB e InfluxDB gravam; projeções leem.



CAMADA 3 Escrita em MariaDB e InfluxDB; leitura em projeções que alimentam Grafana e gêmeo.

Seis módulos em três linguagens; cinco descartes.

MÓDULO	STACK · PAPEL
`proto-shared`	**.proto · gRPC** Contrato canônico
`event-hub-ms-rust`	**Rust · Transactional Outbox** Barramento, caminho quente
`live-ms-java`	**Java 25 · Spring Boot 4** Adaptadores AIS · OlhoVivo · CGESP
`virtual-ms-java`	**Java · Vaadin** Gêmeo digital do campus
`constructive-ms-python`	**Python · AirSim · Ollama/NIM** Missões do drone
`infra-local-shared`	**Docker Compose** Ambiente reprodutível

ALTERNATIVAS DESCARTADAS

× **RTI Portico**
é aberta, mas a curva OMT/FOM pesa

× **Kafka**
operação de cluster pesada para o escopo

× **REST síncrono**
não oferece log nem replay por si só

× **Monolito Java**
forçaria Java onde AirSim e o LLM vivem em Python

× **Banco compartilhado**
acoplaria escrita e leitura, contra o CQRS

Proposta vs. HLA/DIS em cinco critérios.

	HLA / DIS	PROPOSTA
Custo de adoção	RTIs abertas e comerciais existem; exige escolher a RTI e o FOM	Componentes centrais abertos; exige operar o Event Hub e a stack
Modelo de dados	HLA: FOM formal, semântica compartilhada. DIS: PDUs fixas	.proto próprio e flexível, sem semântica padronizada
Integração web e cloud	APIs padronizadas em C++ e Java; o resto, via ponte	gRPC e Grafana diretos; stubs Java, Python e Rust do mesmo .proto
Observabilidade	HLA: MOM e loggers. DIS: captura de PDUs. Painéis exigem adaptador	Projeções de leitura alimentam o Grafana; sem métricas de latência
Maturidade e garantias	Padrões IEEE: gestão de tempo, escala e interoperabilidade entre fornecedores	**Protótipo: at-least-once, sem gestão de tempo, sem teste de escala**

Comparação qualitativa dos autores: sem medição lado a lado nem validação estatística. Não substitui HLA/DIS.

03

PRÓXIMA SEÇÃO

Cenários e avaliação

Dados reais viram eventos de check-in e check-out.

ADAPTADOR 01

AIS

Navios no canal · REST autenticada

ADAPTADOR 02

SPTrans OlhoVivo

Ônibus em tempo real · API com cadastro

ADAPTADOR 03

CGESP

Alagamentos de SP · web scraping

area	entity_id	entity_type	event_type	lvc
Santos Port	710009040	vessel	CHECKIN	LIVE
Santos Port	405000377	vessel	CHECKIN	LIVE
Rua da Consolação	81512	bus	CHECKOUT	LIVE
Rua da Consolação	81512	bus	CHECKIN	LIVE
Consolação Neighborhood	81512	bus	CHECKIN	LIVE
Consolação Neighborhood	81506	bus	CHECKOUT	LIVE
Caio Prado C/B Stop Area	81506	bus	CHECKOUT	LIVE

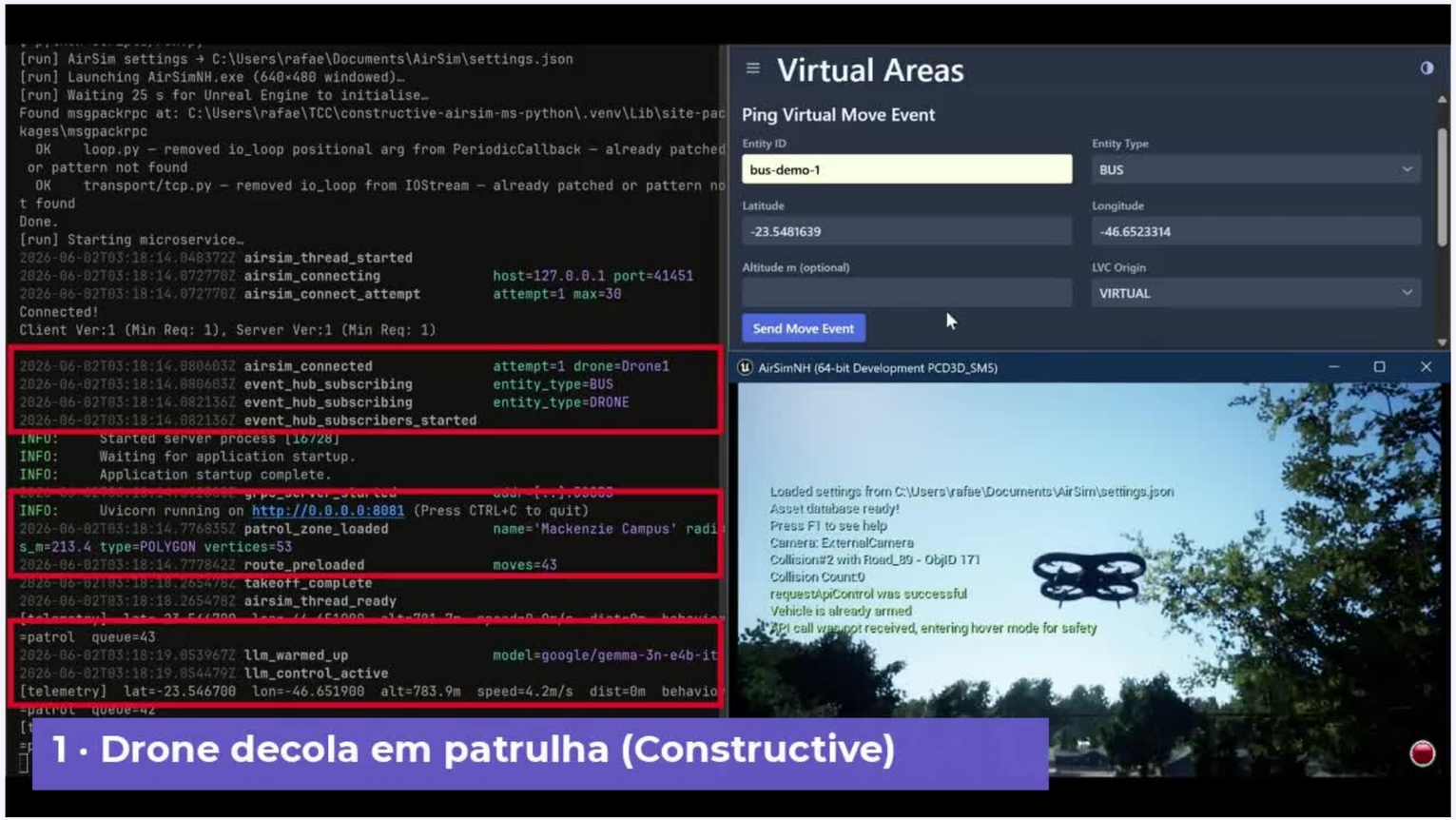
EVENT LOG REAL

Só eventos **LIVE** das fontes públicas.

CHECKIN = entrou numa área do gêmeo.

CHECKOUT = saiu da área.

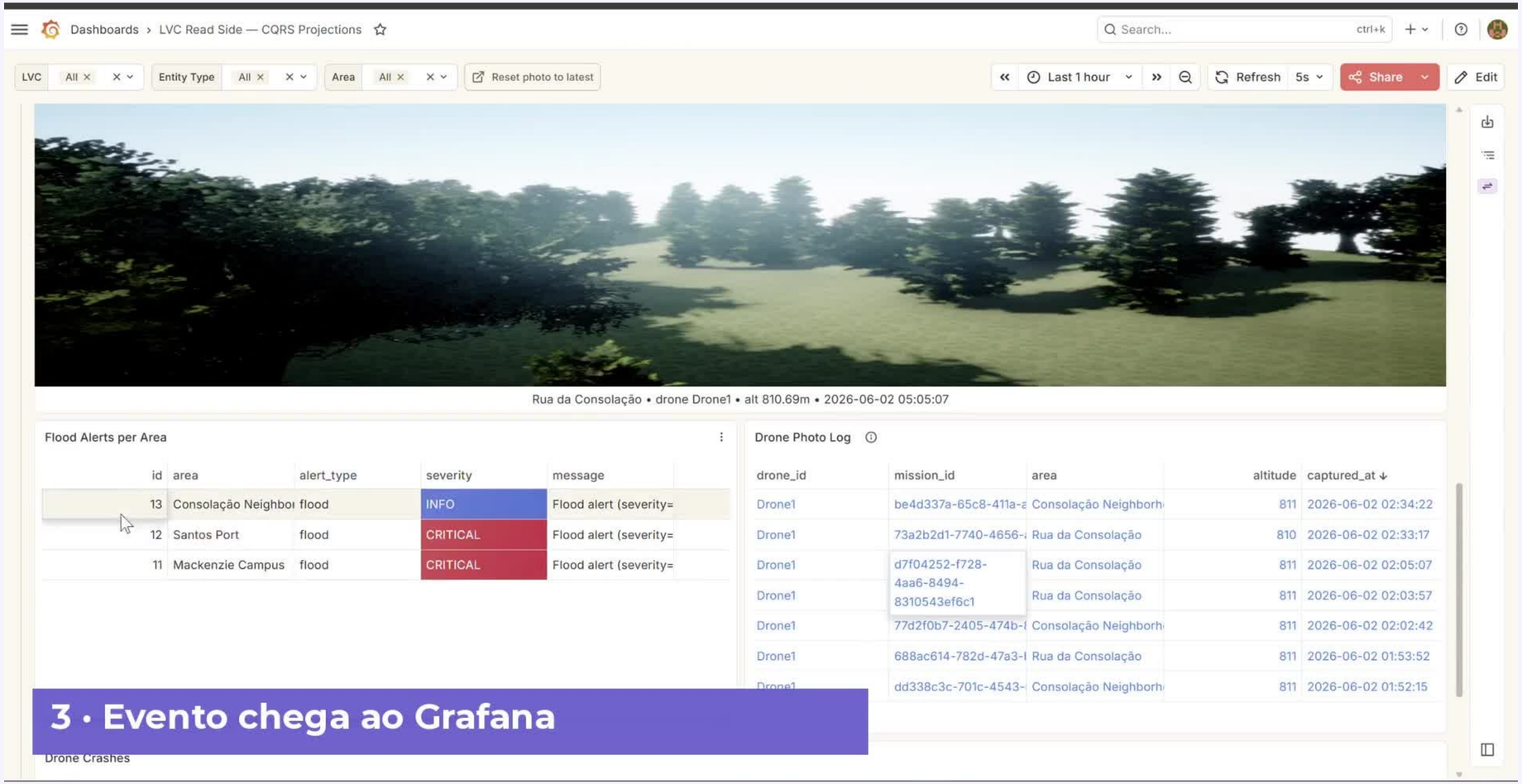
Evento de ônibus (sintético na demo) aciona o drone.



ENCADEAMENTO

1. O drone já patrulha no AirSim (Constructive).
2. Evento de ônibus no ponto Caio Prado; **na demo, sintético (esquema do OlhoVivo).**
3. virtual-ms-java detecta a área e publica o gatilho.
4. O LLM planeja a trajetória; o drone voa e fotografa.
5. Cada passo volta ao Event Hub.

Um alagamento simulado vira alerta no Grafana.



Grafana lê as projeções (read path) em tempo real · vídeo de 30 s com legendas por etapa

Quatro critérios verificados funcionalmente.

Encadeamento LVC

Evento de ônibus (sintético na demo) aciona, via gêmeo, a missão do drone.

`onde observar · cenário 3 · slide 15`

Registro de eventos

Os eventos Live ficam registrados no log do Event Hub.

`onde observar · Event Log · slide 14`

Subida com docker compose

A arquitetura sobe com docker compose up.

`onde observar · infra-local-shared`

Stubs gerados do .proto

Um .proto gera stubs Java, Python e Rust.

`onde observar · proto-shared`

Verificação funcional: latência e escala ficam para trabalho futuro.

04

PRÓXIMA SEÇÃO

Contribuições e limites

Contribuições: arquitetura, contrato, fontes, cenários.

01

Arquitetura event-driven aberta

Para LVC em gêmeos urbanos; complementa HLA/DIS, sem substituí-los.

02

proto-shared: um contrato único

Padrão de contrato canônico para plataformas poliglotas.

03

Três fontes Live reais

AIS, SPTrans OlhoVivo e CGESP, todas públicas.

04

Quatro cenários ponta a ponta

Áreas no gêmeo, check-in Live, missão do drone (gatilho sintético) e observabilidade.

Limites: máquina única, sem validação estatística.

Latin.Science 2026

LIMITAÇÕES

- Sem validação **estatística** Live vs. V/C.
- Sem teste de **escala horizontal**: máquina única.
- Demo do cenário 3 com **gatilho sintético**.
- Sem **idempotência** no consumo nem id de correlação.

PRÓXIMOS PASSOS

- Fidelidade do gêmeo (Live vs. V/C).
- Carga, latência e escala horizontal.
- Idempotência e correlação de eventos.
- Novos adaptadores Live e planejadores.

Repositório aberto: github.com/tcc-modern-lvc-arch

Dúvidas?

Contato:

rafael.informa@gmail.com

LinkedIn

github.com/tcc-modern-lvc-arch
deepwiki.com/tcc-modern-lvc-arch

Projeto e repositório

Latin.Science 2026

Realização:

itaipu parquetec

ITAIPU BINACIONAL